

UNITED STATES PATENT AND TRADEMARK OFFICE (USPTO)

PROVISIONAL PATENT APPLICATION SPECIFICATION

TITLE OF THE INVENTION:

ZERO-TRUST DATA SANITIZATION (ZTDS) ARCHITECTURE FOR EPHEMERAL IN-MEMORY PII REDACTION AND CRYPTOGRAPHIC ENCLAVE ATTESTATION IN ARTIFICIAL INTELLIGENCE WORKFLOWS

INVENTOR:

Ilya Sibiryakov (Tel Aviv, Israel)

CORRESPONDENCE ADDRESS:

Ilya Sibiryakov

support@privacyscrubber.com

Tel Aviv, Israel

CROSS-REFERENCE TO RELATED DISCLOSURES

This application claims the benefit of and formal priority from the inventor's earlier technical disclosures:

1. Sibiryakov, I. *Zero-Trust Data Sanitization Protocol for Generative AI Workflows*, CERN / Zenodo, 2026. DOI: 10.5281/zenodo.22058770.
 2. Sibiryakov, I. *Comparative Latency and Memory Profiling of Client-Side PII Sanitization*, Open Science Framework (OSF), 2026. DOI: 10.17605/OSF.IO/5BYJF.
 3. Sibiryakov, I. *Statutory Compliance in Enterprise Generative AI Under the EU AI Act and GDPR*, Elsevier SSRN, 2026. SSRN ID: 7335581.
 4. Sibiryakov, I. *Preserving Attorney-Client Privilege in Generative AI Workflows via Client-Side ZTDS*, Law Archive / OSF Preprints, 2026. DOI: 10.17605/OSF.IO/4WC86.
-

ABSTRACT

An endpoint-first, hardware-attested zero-trust data sanitization (ZTDS) system and method for enterprise artificial intelligence (AI) workflows is disclosed. The system eliminates cloud Data Loss Prevention (DLP) honeypot liabilities and plaintext transmission over untrusted networks. A client-side volatile

sanitization runtime executing in an isolated local memory environment (e.g., dedicated Web Worker thread) intercepts unstructured enterprise text prior to network serialization. The runtime compiles Deterministic Finite Automata (DFA) patterns corresponding to domain-specific entities (e.g., personal identifiers, healthcare records, financial entries, authentication secrets) and replaces detected sensitive strings with unique, reversible tokens in strict descending order of character offset, preventing index drift. Reversible mapping pairs are maintained exclusively within volatile client RAM bound to the ephemeral lifecycle of an active session or process, guaranteeing mathematical zero-persistence upon termination.

Outbound sanitized payloads are dispatched to enterprise confidential processing sites gated by a Policy Decision Point (PDP) and Attestation Verifier (AV). Downstream execution is conditioned upon cryptographic hardware attestation (e.g., AWS Nitro Enclaves) validating platform identity, expected PCR measurements, and fresh nonces under fail-closed authorization. Following confidential processing, tokenized model responses are deterministically rehydrated at the client endpoint via local in-memory reverse lookup without exposing plaintext or mapping keys to intermediate network hops or cloud providers. In collaborative embodiments, the system provides zero-server cryptographic team session handoff: the volatile session map is serialized and encrypted on a first client endpoint using an authenticated cipher (XChaCha20-Poly1305 AEAD) with a symmetric key derived via a memory-hard password hashing function (Argon2id) and dispatched to a second client endpoint via a client-side URI fragment identifier or standalone encrypted container, enabling cross-endpoint rehydration without intermediate server persistence or database storage.

TECHNICAL FIELD

The present disclosure relates generally to computer system security, data privacy, and confidential computing. More specifically, embodiments of the invention relate to methods, architectures, and computer-readable media for endpoint-native data de-identification, ephemeral in-memory tokenization, cryptographic hardware attestation, zero-server cryptographic session handoff, and secure workflow dispatch for generative artificial intelligence models.

BACKGROUND OF THE INVENTION

Enterprise integration of Large Language Models (LLMs) and generative AI agents across customer service, software development, financial auditing, and legal discovery creates acute data leakage liabilities. Employees routinely input unstructured prompts containing Personally Identifiable Information (PII), Protected Health Information (PHI), proprietary financial records, and cryptographic credentials directly into public or third-party cloud model endpoints.

Conventional data loss prevention approaches rely on intermediate network proxy appliances (e.g., centralized cloud DLP services or SaaS tokenization vaults). These architectures suffer from critical structural deficiencies:

1. **Third-Party Honey-pot Liabilities:** Routing raw enterprise data through intermediary DLP servers aggregates massive volumes of sensitive plaintext in third-party environments, expanding the regulatory attack perimeter under GDPR Article 28 (Processor contracts) and HIPAA (Business Associate Agreements). Compromise of the proxy or transit channel exposes the underlying enterprise assets.
2. **Transit Latency and Overhead:** Centralized proxies introduce synchronous WAN hops, TLS termination/re-encryption overhead, and remote queueing delay, frequently imposing 150–500 ms of latency that degrades real-time interactive AI completions and IDE streaming.
3. **Loss of Contextual Reversibility:** Centralized tokenization architectures require persistent cloud databases (token vaults) to maintain mapping associations, violating zero-retention mandates and creating perpetual insider-threat vulnerabilities.
4. **Lack of Hardware Verification:** Existing DLP systems do not cryptographically verify whether downstream AI processing nodes are executing unmodified, attested firmware or whether data processing is confined to isolated hardware enclaves.

Accordingly, there is a profound unmet need for an endpoint-first sanitization architecture that eliminates cloud proxy exposure, retains reversible mappings strictly in ephemeral client memory, cryptographically couples data release with hardware-attested enclave execution, and enables zero-server collaborative session handoff across enterprise endpoints.

SUMMARY OF THE INVENTION

The present invention solves these technical problems by providing a Zero-Trust Data Sanitization (ZTDS) architecture operating across three distinct planes: (1) an Endpoint Volatile Sanitization Plane, (2) a Policy Decision and Hardware Attestation Plane, and (3) an Attested Confidential Execution Plane.

In an exemplary embodiment, a method comprises:

1. Intercepting, by an endpoint runtime operating in volatile random access memory (RAM), an input query comprising unstructured text prior to network serialization;
2. Identifying, using compiled deterministic finite automata (DFA) matching rules, one or more protected sensitive data entities within the input query;
3. Generating, for each identified protected entity, a unique replacement token;

4. Substituting each identified protected entity with its corresponding replacement token in descending order of starting character offsets to generate a sanitized query, while storing a bijective association between each replacement token and its corresponding plaintext entity exclusively in an ephemeral, tab-isolated or process-isolated in-memory session map;
5. Acquiring signed cryptographic attestation evidence from a target hardware execution enclave (e.g., AWS Nitro Enclave), the attestation evidence comprising platform measurements (PCR0, PCR1, PCR2) and an ephemeral nonce;
6. Cryptographically verifying the attestation evidence and validating that the enclave measurements match authorized policy values;
7. Transmitting the sanitized query to the hardware execution enclave over an isolated virtual communication channel (e.g., virtio-vsock) only upon successful verification of the attestation evidence under a fail-closed policy;
8. Receiving, at the endpoint runtime, a tokenized output response from the hardware execution enclave;
9. Deterministically restoring the tokenized output response to a rehydrated response by substituting each replacement token with its corresponding plaintext entity retrieved from the ephemeral in-memory session map, wherein the ephemeral in-memory session map is destroyed upon session termination without writing plaintext to non-volatile storage; and
10. In a collaborative team embodiment, serializing the ephemeral in-memory session map at a first client device; deriving a symmetric encryption key from an enterprise passphrase using a memory-hard password hashing function (Argon2id); encrypting the serialized session map using authenticated encryption with associated data (XChaCha20-Poly1305 AEAD) with an ephemeral nonce and salt into an encrypted handoff container; dispatching the container to a second client device via a client-side URI fragment identifier; and importing, decrypting, securely zeroing key memory, and rehydrating tokenized AI responses at the second client device in volatile random access memory without intermediate server persistence.

BRIEF DESCRIPTION OF THE DRAWINGS

- **FIG. 1** is a high-level system architecture diagram illustrating the three operational planes of the ZTDS framework: the Client Endpoint ($\text{\$}_{\text{\{RAM\}}}$), the Attestation Verifier and Policy Decision Point (AV/PDP), and the Hardware-Attested Confidential Enclave sites.
- **FIG. 2** is a sequence flow diagram illustrating the 7-phase hardware attestation and fail-closed workload authorization protocol between the client, PDP, and AWS Nitro Enclave.
- **FIG. 3** is a schematic block diagram illustrating the client-side DFA tokenization engine, descending character offset ordering, and volatile heap isolation.

- **FIG. 4** is a comparative performance chart illustrating steady-state throughput and latency scaling of ZTDS versus a centralized HTTPS DLP proxy across 1 KiB, 4 KiB, 16 KiB, and 64 KiB payloads.
- **FIG. 5** is a validation matrix illustrating held-out entity detection precision, recall, and F1 across W-2/payroll, EHR, and code-log synthetic enterprise corpora.
- **FIG. 6** is a cross-platform hardware scaling diagram comparing ZTDS latency and throughput across Intel Xeon (C6i) and AMD EPYC (C6a) hardware enclave families.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

1. Architectural Overview and Operational Planes

Referring to FIG. 1, the ZTDS architecture separates enterprise AI workflows into three decoupled trust zones:

- **Client Endpoint (\$E\$):** An enterprise workstation, browser tab, or IDE runtime executing user tasks. Within the client endpoint, a volatile sanitization runtime ($\$S_{\text{RAM}}\$$) executes inside an isolated memory sandbox (e.g., a Web Worker or isolated process).
- **Attestation & Policy Control Plane (\$AV, PDPS):** An Attestation Verifier (\$AV\$) and Policy Decision Point (\$PDPS\$) responsible for authenticating hardware enclave identities, verifying measurement registers, validating cryptographic nonces, and binding requests to time-limited authorization sessions.
- **Enterprise Confidential Sites (\$C_j\$):** Isolated hardware enclaves (e.g., AWS Nitro Enclaves) hosting private AI models, retrieval-augmented generation (RAG) pipelines, or secure proxy connectors to external foundation LLMs (\$A\$). Enclaves communicate exclusively via local virtual sockets (AF_VSOCK) without external network interfaces.

2. Client-Side DFA Tokenization and Ephemeral Heap Scoping

The client runtime operates on the principle that sensitive plaintext must never cross the network perimeter. When a user or application initiates a prompt or document submission:

1. **Automata Rule Evaluation:** Unstructured text is evaluated against compiled DFA regular expressions representing sensitive entity classes (e.g., `EMAIL`, `PHONE`, `SSN`, `EIN`, `CREDIT_CARD`, `IBAN`, `IPV4`, `API_KEY`, `MRN`, `NAME`).
2. **Descending Offset Substitution:** Detected entity intervals $\$(s_i, e_i)\$$ are sorted in descending order of starting offset ($\$(s_i \geq s_{i+1})\$$). Replacements are executed from the end of the string toward the beginning. This eliminates index drift and invalidation of subsequent match coordinates.

3. **Bijective In-Memory Mapping:** Each distinct plaintext entity x is mapped to a unique surrogate token $t = [\text{TYPE_N}]$ via an ephemeral session dictionary $\mathcal{M}_{\text{session}}$. The relationship is strictly bijective within the active session: $f_s: x \mapsto t, \quad f_s^{-1}: t \mapsto x$
4. **Volatile Scoping and Non-Persistence:** The dictionary $\mathcal{M}_{\text{session}}$ is allocated strictly in volatile RAM. No entries are written to disk, browser LocalStorage, IndexedDB, cookies, or telemetry pipelines. When the browser tab, IDE window, or local process terminates, the memory space is reclaimed by the operating system garbage collector, ensuring that: $\Pr[\text{Plaintext Recovery} \mid \text{Post-Session}] = 0$

3. Cryptographic Hardware Attestation Protocol

To prevent sanitized enterprise payloads from reaching unauthorized or tampered cloud endpoints, ZTDS integrates hardware-backed remote attestation. Referring to FIG. 2, the protocol proceeds in seven distinct phases:

1. **Session Nonce Initialization:** The client generates a cryptographically secure 128-bit pseudorandom nonce $N_s \in \{0,1\}^{128}$ and issues an authorization request to the PDP.
2. **Attestation Challenge:** The PDP generates an enclave-specific challenge nonce n_e and dispatches it to the confidential site C_j over a secure control channel.
3. **Hardware Attestation Evidence Generation:** The Nitro Enclave interacts with the on-chip Nitro Security Module (NSM) via local vsock (`/dev/nsm`). The NSM produces a signed CBOR Object Signing and Encryption (COSE) Sign1 document containing:
 - PCR0: SHA-384 hash of the enclave kernel, application binary, and runtime image (EIF).
 - PCR1: SHA-384 hash of the parent OS kernel and bootstrap configuration.
 - PCR2: SHA-384 hash of the application memory configuration and attached devices.
 - User Data / Ephemeral Public Key: Cryptographic public key generated inside enclave RAM.
 - Nonce: Fresh challenge nonce n_e . The document is digitally signed using an elliptic curve signature (ES384) rooted in the AWS Nitro Attestation Public Key Infrastructure.
4. **Fail-Closed Attestation Verification:** The PDP verifies the COSE signature against the trusted root certificate, confirms freshness of n_e to prevent replay attacks, and asserts that: $\text{PCR0} = \text{PCR0}_{\text{authorized}}, \quad \text{PCR1} = \text{PCR1}_{\text{authorized}}, \quad \text{PCR2} = \text{PCR2}_{\text{authorized}}$. If any check fails, the PDP immediately aborts the session under a fail-closed rule.
5. **Workload-Bound Authorization Session:** Upon successful verification, the PDP provisions a short-lived authorization token τ_{auth} with a strict Time-To-Live (TTL, e.g., 60 seconds). The token cryptographically binds: $\tau_{\text{auth}} = \text{HMAC}\{K_{\text{master}}\}(\text{Site_ID} \parallel \text{Request_ID} \parallel \text{Policy_Version} \parallel \text{Expiry})$

6. **Confidential Dispatch:** The client dispatches the sanitized query Q_T (devoid of plaintext PII) to the enclave over `AF_VSOCK`. The enclave executes the AI inference or retrieval pipeline in hardware-isolated memory.
7. **Lossless Reverse Rehydration:** The enclave returns the tokenized completion R_T to the client endpoint. The client runtime performs a single-pass string replacement, substituting surrogate tokens $[TYPE_N]$ with their original plaintext values from $\mathcal{M}_{\text{session}}$, producing the final rehydrated output R_X .

4. Zero-Server Cryptographic Team Session Handoff Protocol (Argon2id + XChaCha20-Poly1305 AEAD)

To support collaborative multi-user workflows without introducing centralized server-side storage, token databases, or honeypot liabilities, the ZTDS architecture provides a zero-server cryptographic session handoff mechanism:

1. **In-Memory Session Serialization:** A first client endpoint runtime serializes the ephemeral dictionary $\mathcal{M}_{\text{session}}$ and associated token label metadata into an unpersisted binary or structured JSON payload P . Optionally, the payload is compressed in volatile memory via LZ-based algorithms to minimize transfer size.
2. **Memory-Hard Key Derivation (Argon2id):** The first client endpoint generates a cryptographically secure 16-byte random salt S_{salt} $\xrightarrow{\text{gets}} \{0,1\}^{128}$. A 256-bit symmetric key K_{handoff} is derived from a shared team passphrase or ephemeral key exchange using the Argon2id memory-hard password hashing algorithm (version 0x13): $K_{\text{handoff}} = \text{operatorname{Argon2id}}(P_{\text{passphrase}}, S_{\text{salt}}, \text{opslimit}, \text{memlimit})$. Argon2id provides robust cryptographic defense against trade-off attacks, custom ASIC pipelines, and GPU-accelerated parallel brute-force cryptanalysis.
3. **Authenticated Encryption (XChaCha20-Poly1305 AEAD):** The first client generates a cryptographically secure 24-byte random nonce N_{pub} $\xrightarrow{\text{gets}} \{0,1\}^{192}$. The payload P is encrypted using extended-nonce ChaCha20 combined with the Poly1305 authenticator (XChaCha20-Poly1305 IETF construction): $C, T = \text{operatorname{XChaCha20-Poly1305-Encrypt}}(K_{\text{handoff}}, N_{\text{pub}}, P)$. The 192-bit nonce size guarantees collision-free encryption even under decentralized random nonce generation, eliminating nonce-reuse catastrophic failures.
4. **Self-Contained Encrypted Container Packing:** The output container is packed into a structured binary format: $\text{Container} = [V \parallel S_{\text{salt}} \parallel N_{\text{pub}} \parallel C \parallel T]$ where V is a 1-byte version identifier (e.g., $V=0x04$), S_{salt} is the 16-byte salt, N_{pub} is the 24-byte nonce, and $C \parallel T$ is the ciphertext with the 16-byte Poly1305 authentication tag.

5. **Zero-Server Transport Channels:** The container is encoded into a URL-safe Base64 string and exported via:
 - A client-side URI fragment identifier (e.g., `https://domain/#handoff=<ciphertext>`). Under RFC 3986, URI fragments are processed exclusively within client user-agents and are strictly withheld from HTTP request transmissions, preventing exposure to intermediate network proxies, reverse proxies, Web Application Firewalls (WAFs), or web server access logs; or
 - A standalone ephemeral file container (`.pssession`) transferred via out-of-band enterprise channels.
6. **Recipient Endpoint Decryption and Invariant Memory Erasure:** When the second client endpoint ingests the URI fragment or container:
 - The recipient client runtime parses the version header, salt, nonce, and ciphertext;
 - The recipient derives K_{handoff} using Argon2id with the identical parameters and the shared passphrase;
 - The recipient decrypts and verifies authenticity under a fail-closed rule: if the Poly1305 tag does not verify, execution aborts immediately;
 - Upon successful verification, the decrypted session map M_{session} is loaded directly into the volatile RAM of the recipient's isolated execution context;
 - The derived key K_{handoff} and intermediate buffers are immediately zeroed in memory (`memzero(K_{\text{handoff}})`); and
 - The recipient endpoint executes local rehydration of tokenized AI model completions using the imported in-memory session map without any query, record, or credential ever having touched an external server.

5. Formal Security Propositions

The system satisfies seven formal propositions:

- **P1 (Detected-Entity Non-Egress):** Every sensitive entity $x \in \mathcal{D}(Q)$ detected by the client sanitizer is absent from outbound network payloads: $\forall x \in \mathcal{D}(Q), x \notin M_{\text{out}}$.
- **P2 (Session-Mapping Non-Egress):** The reversible token map is never transmitted over external channels in plaintext: $\text{Export}(M_s) = \text{varnothing}$.
- **P3 (Token Injectivity):** Within a session, distinct entities map to distinct tokens: $x_a \neq x_b \implies f_s(x_a) \neq f_s(x_b)$.
- **P4 (Application Non-Persistence):** The session mapping is not persisted to secondary storage: $\text{Persist}(M_s) = \text{varnothing}$.

- **P5 (Fail-Closed Attested Dispatch):** Protected workloads are released to confidential sites only upon valid attestation satisfying policy measurements: $\text{operatorname{Release}}(Q_T, C_j) \implies \text{operatorname{Verify}}(E_e, n_e, \text{Pi}_{\text{PCR}}) = \text{TRUE}$.
- **P6 (Context-Bound Authorization):** Authorization tokens are strictly bound to site, request, and policy contexts, preventing cross-tenant or replayed execution.
- **P7 (Session-Handoff Zero-Server Confidentiality and Integrity):** For any session handoff payload SH , no plaintext entity or mapping association is accessible to intermediate transport nodes: $\Pr[\text{Plaintext Compromise} \mid \text{Server Transit}] = 0$.

6. Empirical Performance and Scalability

Evaluation across 1,200 latency trials, 250,668 steady-state ZTDS operations, and 167,331 centralized DLP proxy operations demonstrates the substantial superiority of client-side ZTDS:

- **Interactive Payload Speedup:** At 1 KiB, ZTDS achieves a mean latency of 0.553 ms and 1,792 ops/sec, compared to 0.929 ms and 1,046 ops/sec for a reference centralized DLP proxy—delivering **1.71 $\times$ throughput**.
- **Medium Payload Speedup:** At 4 KiB, ZTDS achieves 1.360 ms versus 1.768 ms—delivering **1.32 $\times$ throughput**.
- **Large Payload Parity:** At 16–64 KiB, performance converges as payload processing dominates transport overhead.
- **Detector Accuracy:** On a frozen held-out corpus of 300 synthetic enterprise documents comprising 1,925 annotated entities, the configured DFA profiles achieved **Precision = 1.000, Recall = 1.000, F1 = 1.000** with zero detected plaintext leakage.
- **Cross-Platform Generalizability:** Replication across Intel Xeon (C6i) and AMD EPYC (C6a) instances confirmed identical scaling and operational stability across processor architectures.

COMPUTER PROGRAM LISTING APPENDIX (SOURCE CODE REDUCTION TO PRACTICE)

APPENDIX OVERVIEW

This appendix forms an integral part of the patent specification and provides reproducible source code embodiments demonstrating reduction to practice of:

1. **Module 1:** Endpoint Volatile Sanitization & Rehydration Engine (`engine.py`)
2. **Module 2:** Attestation Verifier & Nonce Lifecycle Controller (`verifier.py`)
3. **Module 3:** Hardware Enclave Service & virtio-vsock Handler (`enclave_app.py`)

4. Module 4: Zero-Server Cryptographic Session Handoff Engine (crypto_handoff.js)

MODULE 1: Endpoint Volatile Sanitization & Rehydration Engine (engine.py)

```
"""
ZTDS Client Volatile Sanitization Runtime (S_RAM) - Python Reference Engine
Executes in-memory DFA tokenization and reverse rehydration with zero disk
writes.
"""

import re
from typing import Dict, List, Tuple, Any

REGEX_PATTERNS = {
    'EMAIL': re.compile(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'),
    'PHONE': re.compile(r'\b(?:\+?1[-.\s]?)?\d{2-9}\d{2}\d{3}[-.\s]?'
                        r'\d{4}\b'),
    'SSN': re.compile(r'\b\d{3}-\d{2}-\d{4}\b'),
    'EIN': re.compile(r'\b\d{2}-\d{7}\b'),
    'CREDIT_CARD': re.compile(r'\b(?:\d{4}[-.\s]?){3}\d{4}\b'),
    'IBAN': re.compile(r'\b[A-Z]{2}\d{2}[A-Z0-9]{4}\d{7}(?:[A-Z0-9]?'
                        r'\d{0,16})\b'),
    'IPV4': re.compile(r'\b(?:?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3}'
                        r'(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\b'),
    'API_KEY': re.compile(r'\b(?:sk-[a-zA-Z0-9]{20,48}|AKIA[0-9A-Z]'
                        r'\d{16}|ghp_[a-zA-Z0-9]{36})\b'),
    'MRN': re.compile(r'\bMRN[-\s]?d{6,10}\b', re.IGNORECASE),
    'NAME': re.compile(r'\b(?:Dr\.|Mr\.|Ms\.|Mrs\.)\s+[A-Z][a-z]+'
                        r'(?:\s+[A-Z][a-z]+)?\b')
}

class ZTDSEngine:
    def __init__(self):
        self.session_map: Dict[str, str] = {}      # [TOKEN] -> original
        self.reverse_map: Dict[str, str] = {}      # original plaintext -> [TOKEN]
        self.entity_counters: Dict[str, int] = {}

    def reset_session(self):
        self.session_map.clear()
        self.reverse_map.clear()
        self.entity_counters.clear()
```

```

def tokenize(self, text: str) -> Dict[str, Any]:
    if not text or not isinstance(text, str):
        return {'sanitized_text': '', 'token_count': 0, 'session_tokens':
[]}]

    matches = []
    for entity_type, pattern in REGEX_PATTERNS.items():
        for m in pattern.finditer(text):
            matches.append({
                'start': m.start(),
                'end': m.end(),
                'value': m.group(0),
                'type': entity_type
            })

    # Sort descending by offset to prevent index drift
    matches.sort(key=lambda x: x['start'], reverse=True)

    # Filter out overlapping intervals
    filtered_matches = []
    last_start = float('inf')
    for match in matches:
        if match['end'] <= last_start:
            filtered_matches.append(match)
            last_start = match['start']

    sanitized = text
    for match in filtered_matches:
        token = self.reverse_map.get(match['value'])
        if not token:
            count = self.entity_counters.get(match['type'], 0) + 1
            self.entity_counters[match['type']] = count
            token = f"[{match['type']}]_{count}"
            self.session_map[token] = match['value']
            self.reverse_map[match['value']] = token

        sanitized = sanitized[:match['start']] + token +
sanitized[match['end']:]

    return {
        'sanitized_text': sanitized,
        'token_count': len(filtered_matches),
        'session_tokens': list(self.session_map.keys())
    }

```

```

def rehydrate(self, tokenized_response: str) -> str:
    if not tokenized_response or not isinstance(tokenized_response, str):
        return ''

    rehydrated = tokenized_response
    for token, original_val in self.session_map.items():
        rehydrated = rehydrated.replace(token, original_val)

    return rehydrated

```

MODULE 2: Attestation Verifier & Nonce Lifecycle Controller (`verifier.py`)

```

"""
ZTDS Attestation Verifier (AV) & Nonce Engine
Validates COSE_Sign1 documents, checks PCR measurements against approved
policy,
and verifies freshness of the 128-bit challenge nonce under a fail-closed
policy.
"""

import time
import secrets
import json

APPROVED_PCR0 =
"a1b2c3d4e5f60718293a4b5c6d7e8f90123456789abcdef0123456789abcdef0"
MAX_NONCE_AGE_SEC = 300 # 5 minutes

class AttestationVerifier:
    def __init__(self):
        self.active_challenges = {}

    def generate_challenge(self, site_id: str) -> str:
        nonce = secrets.token_bytes(16)
        nonce_hex = nonce.hex()
        self.active_challenges[nonce_hex] = {
            "site_id": site_id,
            "created_at": time.time()
        }
        return nonce_hex

    def verify_evidence(self, attestation_doc: dict) -> tuple:

```

```

        nonce_hex = attestation_doc.get("nonce_hex")
        if not nonce_hex or nonce_hex not in self.active_challenges:
            return False, "REJECT: Challenge nonce missing or unknown"

        challenge_data = self.active_challenges.pop(nonce_hex)
        age = time.time() - challenge_data["created_at"]
        if age > MAX_NONCE_AGE_SEC:
            return False, f"REJECT: Stale nonce (age: {age:.1f}s > {MAX_NONCE_AGE_SEC}s)"

        pcrs = attestation_doc.get("pcrs", {})
        pcr0 = pcrs.get("pcr0")
        if pcr0 != APPROVED_PCR0:
            return False, f"REJECT: PCR0 mismatch (got {pcr0}, expected {APPROVED_PCR0})"

        return True, "ACCEPT: Attestation evidence verified successfully"

```

MODULE 3: Hardware Enclave Service & virtio-vsock Handler (`enclave_app.py`)

```

"""
ZTDS AWS Nitro Enclave Server
Listens on AF_VSOCK for:
1. Attestation Evidence Request (Port 5000)
2. Sanitized Query Dispatch (Port 5005)
"""

import socket
import json
import os

VSOCK_PORT_ATTESTATION = 5000
VSOCK_PORT_QUERY = 5005

def handle_attestation_request(nonce_bytes):
    """
    Interacts with NSM to generate signed hardware attestation evidence.
    """
    mock_attestation_doc = {
        "module_id": "ztlds-sitel-nitro",
        "timestamp_ms": 1788415000,
        "pcrs": {
            "pcr0":

```

```

"alb2c3d4e5f60718293a4b5c6d7e8f90123456789abcdef0123456789abcdef0",
    "pcr1":
"0000000000000000000000000000000000000000000000000000000000000000",
    "pcr2":
"fedcba9876543210fedcba9876543210fedcba9876543210fedcba9876543210"
    },
    "nonce_hex": nonce_bytes.hex() if isinstance(nonce_bytes, bytes) else
str(nonce_bytes),
    "public_key": "MOCK_ENCLAVE_PUBKEY_SECP384R1",
    "signature": "MOCK_NITRO_HYPERVISOR_SIG"
}
return mock_attestation_doc

def process_query(tokenized_query):
    """
    Executes private enclave workload over tokenized inputs.
    Preserves surrogate tokens intact without accessing external networks.
    """
    return f"ENCLAVE_RESPONSE: Successfully processed prompt:
{tokenized_query}. All private models executed inside hardware enclave."

```

MODULE 4: Zero-Server Cryptographic Session Handoff Engine (`crypto_handoff.js`)

```

/**
 * ZTDS Zero-Server Cryptographic Session Handoff Engine
 * Implements client-side Argon2id key derivation and XChaCha20-Poly1305 AEAD
packaging.
 */

const TEAM_VERSION = 4; // Version 4: Argon2id + XChaCha20-Poly1305

async function packSessionHandoff(sessionMap, passphrase, libsodium) {
    if (!passphrase || passphrase.length < 8) {
        throw new Error('Passphrase must be at least 8 characters.');
```

```

const key = sodium.crypto_pwhash(
  sodium.crypto_aead_xchacha20poly1305_ietf_KEYBYTES,
  passphrase,
  salt,
  sodium.crypto_pwhash_OPSLIMIT_MODERATE,
  sodium.crypto_pwhash_MEMLIMIT_MODERATE,
  sodium.crypto_pwhash_ALG_ARGON2ID13
);

// 3. Serialize session mapping payload
const payload = {
  v: TEAM_VERSION,
  sm: sessionMap,
  ts: Date.now()
};
const plaintext = new TextEncoder().encode(JSON.stringify(payload));

// 4. Encrypt using XChaCha20-Poly1305 authenticated encryption
const ciphertext = sodium.crypto_aead_xchacha20poly1305_ietf_encrypt(
  plaintext,
  null,
  null,
  nonce,
  key
);

// 5. Construct binary container: [Version(1B) | Salt(16B) | Nonce(24B) |
Ciphertext + Tag]
const container = new Uint8Array(1 + salt.length + nonce.length +
ciphertext.length);
container[0] = TEAM_VERSION;
container.set(salt, 1);
container.set(nonce, 1 + salt.length);
container.set(ciphertext, 1 + salt.length + nonce.length);

// 6. Cryptographically wipe key from volatile memory
sodium.memzero(key);

return container;
}

async function unpackSessionHandoff(containerBytes, passphrase, libsodium) {
  const version = containerBytes[0];
  if (version !== TEAM_VERSION) {
    throw new Error('Unsupported container version.');
```

```

    }
    const sodium = libsodium;

    let offset = 1;
    const salt = containerBytes.slice(offset, offset +
sodium.crypto_pwhash_SALTBYTES);
    offset += sodium.crypto_pwhash_SALTBYTES;
    const nonce = containerBytes.slice(offset, offset +
sodium.crypto_aead_xchacha20poly1305_ietf_NPUBBYTES);
    offset += sodium.crypto_aead_xchacha20poly1305_ietf_NPUBBYTES;
    const ciphertext = containerBytes.slice(offset);

    // Derive identical key using Argon2id
    const key = sodium.crypto_pwhash(
        sodium.crypto_aead_xchacha20poly1305_ietf_KEYBYTES,
        passphrase,
        salt,
        sodium.crypto_pwhash_OPSLIMIT_MODERATE,
        sodium.crypto_pwhash_MEMLIMIT_MODERATE,
        sodium.crypto_pwhash_ALG_ARGON2ID13
    );

    try {
        // Authenticated decryption under fail-closed verification
        const decryptedRaw =
sodium.crypto_aead_xchacha20poly1305_ietf_decrypt(
            null,
            ciphertext,
            null,
            nonce,
            key
        );
        const jsonStr = new TextDecoder().decode(decryptedRaw);
        return JSON.parse(jsonStr);
    } finally {
        // Invariant memory wipe
        sodium.memzero(key);
    }
}

```

EXEMPLARY CLAIMS

1. A computer-implemented method for zero-trust data sanitization in artificial intelligence workflows, comprising: intercepting, by an endpoint runtime executing in volatile memory of a client computing device, an input query comprising unstructured text prior to serialization onto an external network; identifying, using one or more pattern recognition automata, one or more sensitive entities within the input query; substituting each identified sensitive entity with a unique surrogate token to generate a sanitized query, while maintaining an association between each surrogate token and its corresponding sensitive entity exclusively within an ephemeral, in-memory session map in volatile memory; validating cryptographic hardware attestation evidence received from a remote hardware-isolated execution enclave, the attestation evidence comprising measurements of software executing within the hardware-isolated execution enclave; transmitting, upon successful validation of the attestation evidence under a fail-closed policy, the sanitized query to the hardware-isolated execution enclave; receiving, from the hardware-isolated execution enclave, a tokenized output response; and rehydrating the tokenized output response at the client computing device by replacing each surrogate token with its corresponding sensitive entity retrieved from the ephemeral, in-memory session map.
2. The method of claim 1, wherein substituting each identified sensitive entity is performed in descending order of starting character offsets within the input query, thereby preventing character offset distortion during string replacement.
3. The method of claim 1, wherein the ephemeral, in-memory session map is strictly isolated to a single browser execution context or application process and is irrevocably destroyed upon closure of the execution context without persisting sensitive entity data to non-volatile storage.
4. The method of claim 1, wherein the cryptographic hardware attestation evidence comprises a cryptographically signed document generated by a hardware security module, the document including a fresh challenge nonce and platform configuration registers (PCR0, PCR1, PCR2) reflecting a cryptographic digest of an enclave image format (EIF).
5. The method of claim 4, wherein validating the attestation evidence comprises confirming that the PCR0 measurement matches a predetermined authorized hash value and that the challenge nonce matches an unexpired challenge issued by a policy decision point.
6. The method of claim 1, wherein communication with the hardware-isolated execution enclave is conducted over a local virtual socket channel (vsock) bypassing external network interfaces.
7. The method of claim 1, wherein the pattern recognition automata comprise a plurality of pre-compiled deterministic finite automata (DFA) corresponding to personally identifiable information (PII), protected health information (PHI), financial account credentials, and software secrets.
8. A secure endpoint data sanitization system, comprising: one or more processors; and a volatile memory coupled to the one or more processors, the volatile memory storing instructions that, when executed by the one or more processors, cause the system to: detect, in unstructured application text, sensitive data entities using compiled automata matchers; replace the detected sensitive data entities

with unique surrogate tokens to construct a sanitized application payload; record mapping associations between the surrogate tokens and the detected sensitive data entities strictly within an ephemeral in-memory dictionary; dispatch the sanitized application payload to an attested confidential hardware enclave only upon verification of enclave attestation evidence against an authorized policy; and rehydrate surrogate tokens in an enclave response using the ephemeral in-memory dictionary.

9. A non-transitory computer-readable storage medium comprising instructions that, when executed by a client computing system, cause the system to perform the method of claim 1.
10. The method of claim 1, further comprising: serializing the ephemeral, in-memory session map at the client computing device; deriving a symmetric encryption key from an enterprise passphrase using a memory-hard password hashing function comprising Argon2id; encrypting the serialized session map using an authenticated encryption with associated data (AEAD) cipher comprising XChaCha20-Poly1305 with an ephemeral random nonce and salt, to construct an authenticated encrypted handoff payload; and transmitting the encrypted handoff payload to a second client computing device via a client-side uniform resource identifier (URI) fragment or encrypted file container, wherein the encrypted handoff payload bypasses intermediate server persistence and database storage.
11. The method of claim 10, further comprising: receiving the encrypted handoff payload at the second client computing device; deriving the symmetric encryption key at the second client computing device using the memory-hard password hashing function; decrypting the encrypted handoff payload into an ephemeral, volatile memory session map of the second client computing device; erasing the derived symmetric encryption key from memory of the second client computing device; and rehydrating tokenized artificial intelligence responses at the second client computing device by replacing surrogate tokens with sensitive entities retrieved from the decrypted ephemeral session map.
12. A zero-server collaborative data sanitization system, comprising: a first client computing device comprising a first processor and a first volatile memory, configured to: sanitize an artificial intelligence query by substituting sensitive entities with surrogate tokens; store associations between the sensitive entities and surrogate tokens in a first volatile in-memory dictionary; derive a symmetric key using Argon2id; encrypt the first in-memory dictionary using XChaCha20-Poly1305 authenticated encryption into an encrypted handoff container; and export the encrypted handoff container via a client-side URI fragment; and a second client computing device comprising a second processor and a second volatile memory, configured to: import the encrypted handoff container from the URI fragment; decrypt the container into a second volatile in-memory dictionary in the second volatile memory; cryptographically wipe the symmetric key; and rehydrate tokenized responses from an artificial intelligence model using the second volatile in-memory dictionary.